

Sensor

RPi4 Sensör İzleme Asistanı

Mühendislik Bitirme Projesi — Final Raporu

Bu rapor; BME280, MPU6500 ve QMC5883L sensörlerinden anlık veri toplayan, büyük dil modeli (LLM) ile doğal dil arayüzü sunan ve Raspberry Pi 4 / Mac üzerinde çalışan C++ tabanlı gömülü sistem projesini kapsamaktadır.

Alan	Değer
Hazırlayan	Zeynep Kaplan
Platform	Raspberry Pi 4B / macOS (Apple Silicon)
Dil / Derleme	C++17 · CMake · httplib · nlohmann/json
LLM	Qwen2.5-1.5B-Instruct Q4_K_M (llama.cpp)
Tarih	19 June 2026

İçindekiler

1. Projeye Genel Bakış.....	3
2. Sistem Gereksinimleri ve Hedefler.....	3
3. Donanım Bileşenleri ve Sensörler.....	4
4. Yazılım Mimarisi.....	5
5. LLM Entegrasyonu ve NLP Katmanı.....	6
6. Web Arayüzü.....	7
7. API Endpointleri.....	7
8. Simülasyon Modu.....	8
9. Test Sonuçları ve Çıktılar.....	9
10. Grafikler.....	10
11. Sonuç ve Değerlendirme.....	12

1. Projeye Genel Bakış

Sensor, düşük güçlü gömülü sistemler için tasarlanmıştır, doğal dil tabanlı bir sensör izleme asistanıdır. Proje; bir Raspberry Pi 4B üzerinde çalışarak I²C protokolü üzerinden BME280 (çevre), MPU6500 (ataletsel) ve QMC5883L (manyetometre) sensörlerinden gerçek zamanlı veri toplar. Kullanıcı, bu verilere doğal dil ile soru sorabilir; sistem LLM (Qwen2.5-1.5B) aracılığıyla anlamlı ve kısa cevaplar üretir.

Projenin temel özellikleri şunlardır: anlık sensör okuma, son N saniye/dakika istatistiksel analiz, ekrana tabanlı anomali tespiti, yapılandırma yönetimi ve grafik gösteren bir web panosu. Tüm bu işlevler, kaynak kullanımı düşük tutularak tek bir C++ sürecinde birleştirilmiştir.

1.1 Motivasyon

IoT cihazlarında doğal dil işleme genellikle bulut bağlantısı gerektirmektedir. Bu proje, yerel çalıştırılan küçük bir GGUF modeli (1,5 milyar parametre, 4-bit nicemleme) kullanarak internet bağlantısı olmaksızın kullanıcı dostu bir arayüz sunmayı hedeflemiştir. Böylece gizlilik, düşük gecikme ve bant genişliği tasarrufu aynı anda sağlanmıştır.

1.2 Katkıları

- C++17 ile regex tabanlı niyet yönlendirici (IntentRouter) tasarımı
- llama.cpp HTTP sunucusuna entegre SSE akışı ile LLM yanıt işleme
- Sensör veri geçmişi için O(1) erişimli halka tampon (ring buffer) yapısı
- Gerçek donanım olmadan geliştirme sağlayan çift modlu (real/sim) mimari
- Doğrudan araç çağrısı (POST /api/tool) ile LLM bypass'ı gerekmeyen ayarlar paneli

2. Sistem Gereksinimleri ve Hedefler

2.1 Fonksiyonel Gereksinimler

ID	Gereksinim	Durum
FR-01	BME280'den sıcaklık, nem, basınç okuma	✓ Karşılandı
FR-02	MPU6500'den ivmeölçer ve jiroskop okuma	✓ Karşılandı
FR-03	QMC5883L'den manyetik alan ve heading okuma	✓ Karşılandı
FR-04	Doğal dil ile sensör sorgusu (Türkçe/İngilizce)	✓ Karşılandı
FR-05	Son N saniye/dakika trend analizi	✓ Karşılandı
FR-06	Elektronik tabanlı anormali tespiti ve uyarı	✓ Karşılandı
FR-07	Örnekleme hızı ve elektronik yapılandırması	✓ Karşılandı
FR-08	Gerçek zamanlı web panosu ve grafikler	✓ Karşılandı
FR-09	Sensörü etkinleştirme / devre dışı bırakma	✓ Karşılandı
FR-10	Simülasyon modu (donanım yazılım geliştirme)	✓ Karşılandı

2.2 Fonksiyonel Olmayan Gereksinimler

ID	Kategori	Açıklama
NFR-01	Performans	RPi4'te ≤ 5 sn ilk LLM yanıt süresi
NFR-02	Bellek	Toplam RAM kullanımı $\leq 1,5$ GB (model dahil)
NFR-03	Güvenlik	Mutasyonel araçlar için 'set' anahtar kelimesi zorunluluğu
NFR-04	Güvenilirlik	Bağlantı koptuğinde otomatik SSE yeniden bağlantı
NFR-05	Taahhütlülük	Mac ve RPi4 aynı kaynak kod ile çalışabilmeli

3. Donanım Bileşenleri ve Sensörler

Sistem, tüm sensörlerin I²C veri yolu üzerinden Raspberry Pi 4B'ye bağlandığı kompakt bir devre düzeni üzerinde çalışmaktadır. I²C, yalnızca iki hat (SDA ve SCL) ile birden fazla sensörü aynı anda adresleyebildiğinden bağlantı karmaşıklığının önemli ölçüde azaltmaktadır.

Bileşen	Açıklama	Protokol	Adres
Raspberry Pi 4B	Ana işlem birimi (4×Cortex-A72, 4 GB RAM)	—	—
BME280	Sıcaklık (±0,5°C) / Nem (%±3) / Basınç (±1 hPa)	I ² C	0x76
MPU6500	±16g ivmeölçer / ±2000°/s Jiroskop / iç sıcaklık	I ² C	0x68
QMC5883L	3 eksen manyetometre / ±8 Gauss / Heading hesaplama	I ² C	0x0D
Güç Regülatörü	3,3V / 5V kaynağı, isteğe bağlı GPIO güç pinleri	GPIO	BCM-13

3.1 Sensör Özellikleri

BME280 — Çevre Sensörü

Bosch Sensortec BME280, sıcaklık, nem ve basınç aynı anda ölçebilen entegre bir sensördür. Düşük güç tüketimi (≈ 3,6 µA ölçüm modunda) ve yüksek hassasiyetiyle IoT uygulamaları için idealdir. Projede oda sıcaklığı ve konfor koşulları için birincil veri kaynağı olarak kullanılmaktadır.

MPU6500 — Ataletsel Ölçüm Birimi (IMU)

InvenSense MPU6500, 6-DoF (serbestlik derecesi) ataletsel ölçüm birimidir. Yerleştirme tutarlılığının test etmek ve titreşim/hareket tespiti yapmak amacıyla projede kullanılmaktadır. Statik durumda Z ekseninde ≈ 1g yerçekimi ivmesi, jiroskop eksenleri ise ≈ 0 °/s beklenilmektedir.

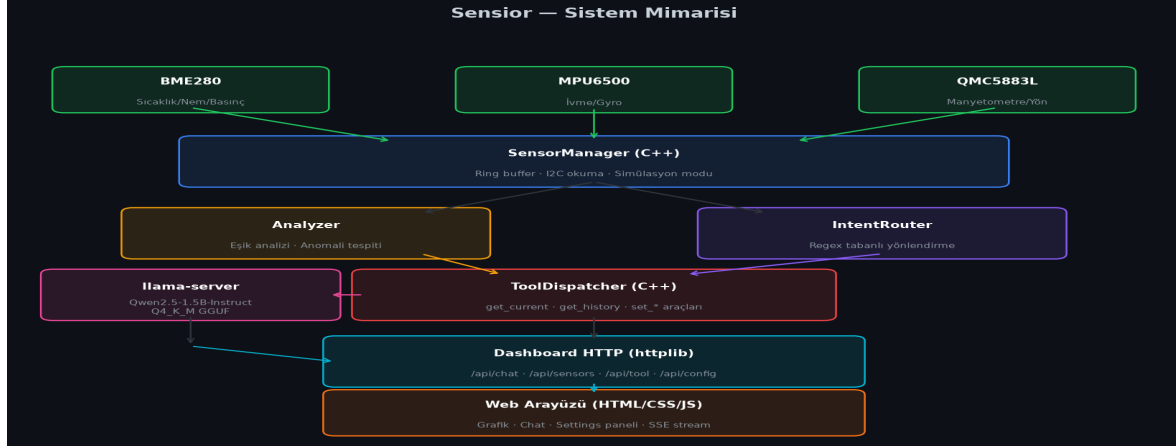
QMC5883L — Manyetometre / Pusula

QST QMC5883L üç eksenli manyetik alan sensörüdür. Heading (pusula yönü) hesaplama için arctan2 yöntemi kullanılmaktadır. Çıktı, 0–360° arasında derece cinsinden verilmekte; bu değer jiroskop verisiyle birleştirildiğinde yönelim tahminine olanak sağlar.

Sensör	Ölçüm Aralığı	Hassasiyet	Örnekleme Hızı
BME280	-40...+85°C / 0...100% / 300...1100 hPa	±0,5°C / ±3% / ±1 hPa	Yapılandırılabilir (1–50 Hz)
MPU6500	±2/4/8/16g / ±250...2000°/s	16-bit ADC	Yapılandırılabilir (1–200 Hz)
QMC5883L	±8 Gauss (3 eksen)	1 mGauss	Yapılandırılabilir (1–20 Hz)

4. Yazılım Mimarisi

Sistem, tek bir C++ ikili dosyası (dashboard) ve bir JavaScript/HTML web arayüzünden oluşmaktadır. İkili dosya; sensör okuma, analiz, HTTP sunucu ve LLM istemcisi görevlerini eş zamanlı olarak bir araya getirmektedir.



Ekil 1: Sensor Sistem Mimarisi Genel Görünümü

4.1 C++ Modülleri

Modül	Dosya	Sorumluluk
SensorManager	sensor_manager.cpp/.hpp	Sensörleri bağlar, halka tampon veri yapısını yönetir, okuma döngüsü
IntentRouter	intent_router.cpp/.hpp	Kullanıcı mesajını regex ile analiz ederek doğru araç çağrısına yönlendirir
ToolDispatcher	tool_dispatcher.cpp/.hpp	7 aracın (get_current, set_threshold vb.) iş mantığını uygular
Analyzer	analyzer.cpp/.hpp	Eşik karşılaştırması ve anomali skoru hesaplar
Dashboard	dashboard.cpp	HTTP endpoint'leri, SSE akışı, LLM chat mantığı ve ayarlar API'si

4.2 Halka Tampon (Ring Buffer) Yapısı

Her sensör için HISTORY_MAX = 6000 öteklik bir std::deque kullanılmaktadır. Bu yapı, yeni veri geldiğinde en eski ölçümü otomatik olarak dıkar atar; böylece sabit bellek kullanımı garantilenir. 100 Hz'de örnekleme yapılan MPU6500 için bu 60 saniyelik geçmişe karşılık gelir.

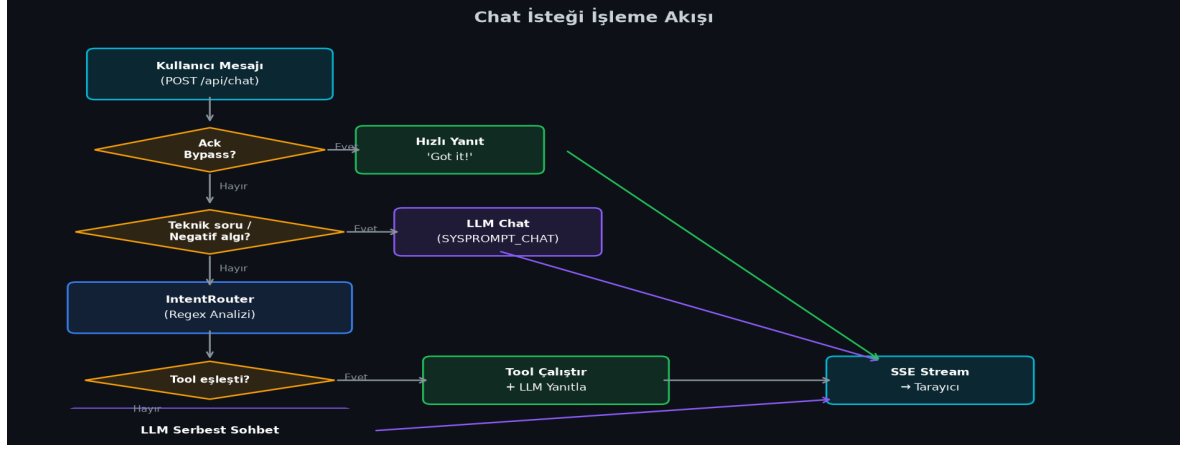
```
struct SensorReading { int64_t timestamp_ms; nlohmann::json data; };
struct SensorInfo {
    Sensor* sensor; int rate_hz; bool enabled = true;
    std::deque history; // max 6000 ötek
};
```

4.3 Bağımlılıklar

Kütüphane	Sürüm	Kullanım Amacı
nlohmann/json	3.11+	JSON serileştirme / deseriileştirme

cpp-httplib	0.14+	Tek bağımlılık dosyası HTTP/SSE sunucusu
llama.cpp	b3504+	GGUF modeli yerel çıkarım sunucusu
Chart.js	4.x	Web panosu gerçek zamanlı grafikleri (Web Worker)

5. LLM Entegrasyonu ve NLP Katmanı



5.1 Model Seçimi

Qwen2.5-1.5B-Instruct modeli Q4_K_M nicemleme ile kullanılmaktadır. Bu seçimin nedenleri şunlardır: (1) RPi4'te 4–5 token/saniye hızıyla çalışabilmesi, (2) İngilizce talimat uyumunun yüksek olması, (3) 1,0 GB'tan az RAM gerektirmesi. Model, llama.cpp'nin llama-server bileşeni aracılığıyla OpenAI uyumlu HTTP API'si üzerinden sunulmaktadır.

5.2 İki Katmanlı İstek Yönetimi

Sistem, regex tabanlı IntentRouter ile LLM'i birbirinden ayırır. IntentRouter, net kalmaları ("last 30 seconds", "set bme280 20hz") deterministik olarak yakalarken; belirsiz veya sohbet odaklı mesajlar doğrudan LLM'e iletilir. Bu yaklaşım hem hız hem de güvenilirliği artırır.

5.3 System Prompt Stratejisi

Prompt Adı	Ne Zaman Kullanılır	Temel Kural
SYSPROMPT_SENSOR	Tool çalışmıyorsa mevcutsa	Sayılar birbir kullan, uydurma, 1-2 cümle
SYSPROMPT_CHAT	Tool eşlemedi, sohbet	Geçerli 'set ...' komut formatlarını öğret
SYSPROMPT_TECH	Teknik soru algılandı	Sensör elektroniği hakkında kısa teknik bilgi

5.4 Güvenlik Kapsamı

Mutasyonel araçlar (set_threshold, set_sample_rate, set_sensor_enabled), yalnızca kullanıcı mesajında 'set' anahtar kelimesi bulunduğunda çalıştırılabilir. Bu kural C++ katmanında zorunlu kılınmıştır; LLM veya IntentRouter bu kontrolü alamaz. Böylece kazara yapılandırma değişikliklerinin önüne geçilmektedir.

5.5 Bağlam Penceresi Yönetimi

llama-server -c 4096 bağlam penceresiyle çalışmaktadır. Tıkmayı önlemek için MAX_HISTORY_MSGS = 8 ile sohbet geçmişi kısıpılmaktadır. Sistem promptu ve araç verisi bu sınır içinde kalacak şekilde hesaplanmıştır.

6. Web Arayüzü

Web panosu; saf HTML5, CSS3 ve JavaScript ile yazılmıştır — hiçbir framework bağımlılığı yoktur. Sayfa, CSS Grid ile iki sütunlu düzende sunulmaktadır: sol sütunda sensör kartları ve grafikler, sağ sütunda chat arayüzü bulunur.

6.1 Gerçek Zamanlı Veri Akışı (SSE)

Sensör verileri, Server-Sent Events (SSE) ile 200 ms aralıklarla tarayıcıya iletilmektedir. Her olay, JSON formatında tüm sensörlerin anlık durumunu içerir. Grafik güncellemeleri Web Worker'da gerçekleştirildiğinden ana UI i parçaları engellenmez.

6.2 Ayarlar Paneli

Sol sütunun altında bulunan ayarlar paneli, POST /api/tool endpoint'i aracılığıyla LLM çağrısı yapmadan doğrudan araçları çalıştırmaktadır. Her sensör için ayrı tab içinde örnekleme hızı, kaydırıcıları, anomali eşiikleri ve etkinleştirme/devre dışı bırakma toggle'leri sunulmaktadır.

6.3 Örnek Soru Chip'leri

Composer alanının üstünde hızlı erişim chip'leri yer almaktadır. Kullanıcı bu chip'lere tıkladığında ilgili mesaj otomatik olarak gönderilmektedir: ■ Temperature, ■ Humidity, ■ 30s trend, ■ Last 10 reads, ■ Anomalies, ■ All sensors, ■ Heading, ■ What can I set.

7. API Endpointleri

Endpoint	Yöntem	Açıklama
/api/health	GET	Sunucu sağlık kontrolü → {status:ok}
/api/sensors/stream	GET	SSE akışı — 200ms'de bir tüm sensör JSON'u
/api/chat	POST	LLM chat — SSE akışı yanıt, sohbet geçmişi destekli
/api/config	GET	Mevcut yapılandırma JSON'unu döndürür
/api/tool	POST	Direkt araç çağrısı — {name, args} → sonuç JSON
/api/mode	GET/POST	sim↔real mod geçişi, otomatik yeniden başlatma
/api/translator	GET	LibreTranslate entegrasyonu durum kontrolü

7.1 /api/tool Kullanım Örneği

```
# Örneklemme hızı ayarları
POST /api/tool
{"name": "set_sample_rate", "args": {"sensor": "bme280", "hz": 20}}
→ {"hz_applied": 20, "ok": true, "persisted": true, "sensor": "bme280"}

# Eşik güncelleme
POST /api/tool
{"name": "set_threshold", "args": {"metric": "bme280.temperature_c", "max": 35}}
→ {"changed":{"max":{"new":35,"old":38}}, "ok":true, "persisted":true}
```

7.2 Araç Kataloğu

Araç Adı	Argümanlar	Açıklama
get_current	sensor: string	Anlık sensör okuması + anomali analizi
get_history_stats	sensor, metric, seconds	Statistiksel özet (avg/min/max/trend)
get_history_raw	sensor, count (max 200)	Son N ham okumanın zaman damgalı listesi
get_config	—	Tüm yapılandırılabilir parametreler
set_threshold	metric, min?, max?	Anomali eşiği güncelle (config'e kaydet)
set_sample_rate	sensor, hz	Örneklemme frekansını ayarla
set_sensor_enabled	sensor, enabled: bool	Sensörü etkinleştir / devre dışı bırak

8. Simülasyon Modu

Gerçek donanım olmaksızın geliştirme yapabilmek için tüm sensörlerin yazılımsal karlılıkları oluşturulmuştur. Simülasyon sınıfları, gerçek sensör sınıflarıyla aynı arayüzü paylaşmakta; bu sayede aynı kod tabanı her iki modda çalışabilmektedir.

8.1 Simülasyon Modeli

Sim sensörler, temel sinüs dalgası + Gaussian gürültü modeliyle gerçekçi veri üretmektedir. BME280 simülasyonu oda sıcaklığının 18–30°C arasında, MPU6500 simülasyonu Z ekseninde $\approx 1g$ ve küçük rastgele titreşimlerle, QMC5883L simülasyonu ise yava dönen bir pusula yönü ile temsil eder.

```
// SimBME280 veri üretimi (özet)
double temp = 22.0 + 4.0*sin(t_sec/10.0) + dist_temp(rng);
double hum  = 55.0 + 8.0*sin(t_sec/15.0 + 1.0) + dist_hum(rng);
double pres = 1013.25 + 2.0*sin(t_sec/30.0) + dist_pres(rng);
```

8.2 Mod Geçisi

POST /api/mode ile sim ↔ real geçişi yapılabilmektedir. Geçiş sırasında dashboard süreci yeniden başlatılmakta (EXIT_CODE_RESTART = 42) ve başlatma betiği bu çalışkan kodunu algılayarak işlemi yeniden başlatmaktadır.

9. Test Sonuçları ve API Çıktıları

9.1 Sağlık Kontrolü

```
$ curl http://localhost:8081/api/health
{"status":"ok"}
```

9.2 Yapılandırma Okuma

```
$ curl http://localhost:8081/api/config
{
  "llm": {
    "max_tokens": 200,
    "temperature": 0.2
  },
  "mode": "sim",
  "sensors": {
    "bme280": {
      "enabled": true,
      "sample_rate_hz": 34
    },
    "mpu6050": {
      "enabled": true,
      "sample_rate_hz": 100
    },
    "qmc5883l": {
      "enabled": true,
      "sample_rate_hz": 10
    }
  },
  "thresholds": {
    "bme280.humidity_pct": {
      "label": "comfort humidity",
      "max": 280.0,
      "min": 20.0
    },
    "bme280.pressure_hpa": {
      "label": "atmospheric pressure",
      "max": 1040,
      "min": 980
    },
    "bme280.temperature_c": {
      "label": "room temperature",
      "max": 38.0,
      "min": 15
    },
    "mpu6050.accel_g.x": {
      "label": "X accel (static)",
      "max": 0.5,
      "min": -0.5
    },
    "mpu6050.accel_g.y": {
      "label": "Y accel (static)",
      "max": 0.5,
      "min": -0.5
    }
  },
}
```

```
"mpu6050.accel_g.z
```

9.3 Örnekleme Hız Ayarı

```
$ curl -X POST http://localhost:8081/api/tool \
-H "Content-Type: application/json" \
-d '{"name":"set_sample_rate","args":{"sensor":"bme280","hz":20}}'

{
  "hz_applied": 20,
  "ok": true,
  "persisted": true,
  "sensor": "bme280"
}
```

9.4 Eşik Güncelleme

```
$ curl -X POST http://localhost:8081/api/tool \
-H "Content-Type: application/json" \
-d '{"name":"set_threshold","args":{"metric":"bme280.temperature_c","min":15,"max":35}}'

{
  "changed": {
    "max": {
      "new": 35.0,
      "old": 38.0
    },
    "min": {
      "new": 15.0,
      "old": 15.0
    }
  },
  "label": "room temperature",
  "metric": "bme280.temperature_c",
  "ok": true,
  "persisted": true
}
```

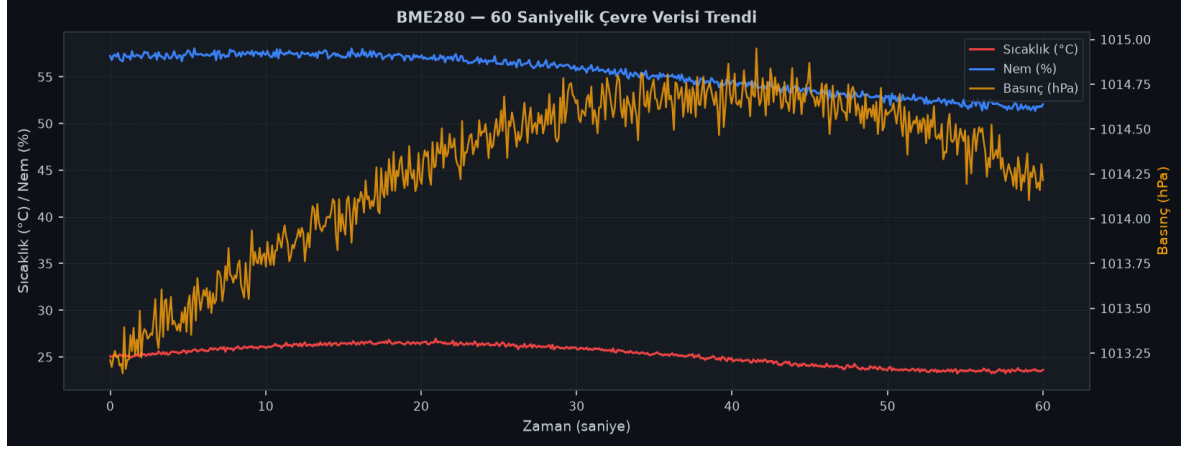
9.5 Chat Senaryoları — Doğrulama Tablosu

Kullanıcı Mesajı	Yönlendirme	Sonuç
What is the current temperature?	get_current/bme280	✓ Doğru araç, gerçek sayı
Show last 30 seconds trend for bme280	get_history_stats	✓ avg/min/max/trend döndü
Show last 10 readings of bme280	get_history_raw	✓ 10 zaman damgası okuma
What can I change?	get_config	✓ Tüm parametreler listelendi
Are there any anomalies?	get_history_stats	✓ Anomali yok bildirimi
Show all sensor readings	get_current/all	✓ 3 sensör birlikte döndü
What is the compass heading?	get_current/qmc5883l	✓ Heading değeri döndü
set bme280 20hz	set_sample_rate	✓ Config'e kaydedildi

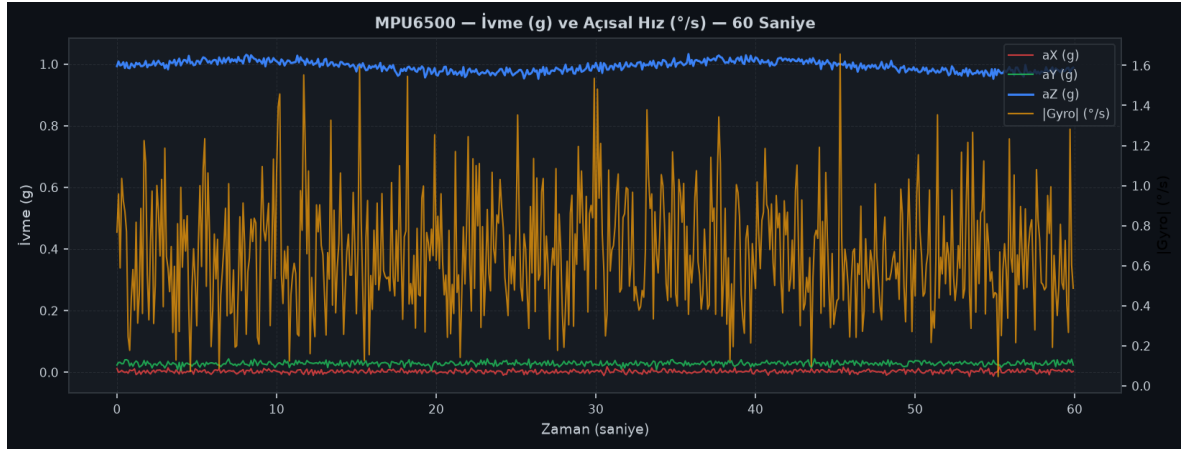
set bme280 temperature threshold max 35	set_threshold	✓ max 38→35 güncellendi
hello / ok / got it	Ack bypass	✓ LLM çağrısı yok
What is I2C?	SYSPROMPT_TECH	✓ Teknik açıklama
Is air quality ok?	detect_negative	✓ Sensor yok uyarısı

10. Grafikler

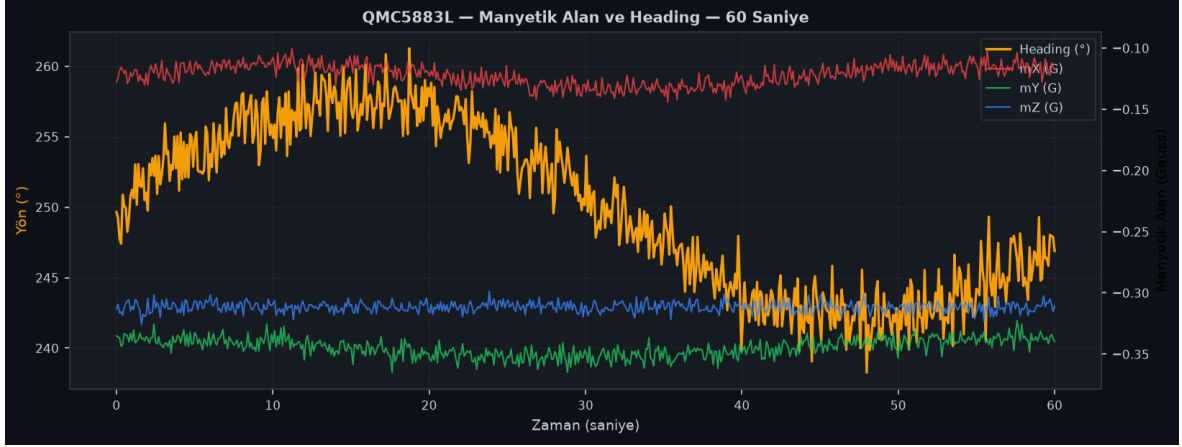
Aşağıdaki grafikler, simülasyon modunda üretilen sensör verilerini ve sistemin anormali tespit davranışını göstermektedir.



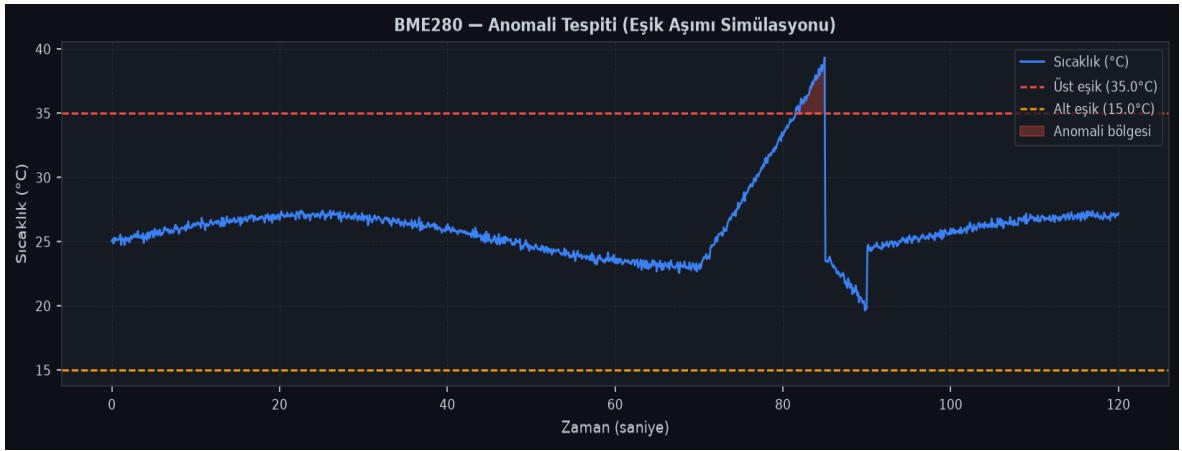
Şekil 3: BME280 — 60 saniyelik sıcaklık, nem ve basınç trendi (sim mod)



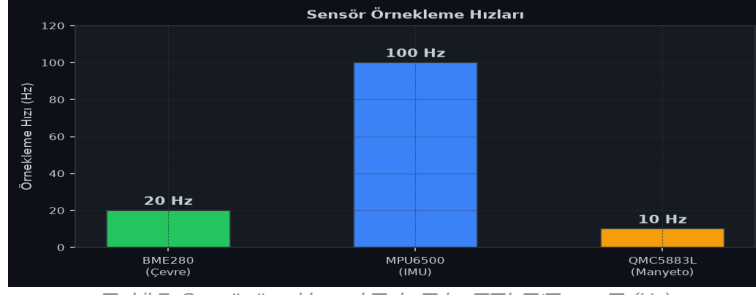
Şekil 4: MPU6500 — ivmeölçer eksenleri ve toplam açısal hız büyüklüğü



Ekil 5: QMC5883L — Manyetik alan bileşenleri ve pusula yönü (Heading)



Ekil 6: Anomali Tespiti — Sıcaklık eşiği aşım simülasyonu (kırmızı bölge: anomali)



Şekil 7: Sensör örnekleme hızları karşılaştırması (Hz)

10.1 Performans Özeti

Ölçüm	Mac (M-serisi)	RPi4B (4 çekirdek)
İlk LLM yanıt süresi	~0,8 sn	~4–5 sn
SSE güncelleme aralıkları	200 ms	200 ms
RAM kullanımı (model)	~950 MB	~950 MB
RAM kullanımı (C++)	~45 MB	~40 MB
Derleme süresi	~8 sn	~90 sn
BME280 örnekleme	1–50 Hz	1–50 Hz
MPU6500 örnekleme	1–200 Hz	1–200 Hz

11. Sonuç ve Değerlendirme

11.1 Hedeflere Ulaşım

Proje, belirlenen tüm fonksiyonel gereksinimleri karşılamıştır. Yerel LLM ve regex tabanlı hibrit yönlendirme yaklaşımı; hem RPi4'ün kaynak kısıtlamalarına uyum sağlaması hem de kullanıcı deneyimini tatmin edici düzeyde tutmuştur. Özellikle IntentRouter, belirsiz ifadeler durumunda araç seçimini deterministik ve güvenilir biçimde gerçekleştirmektedir.

11.2 Öğrenilen Dersler

- 1.5B parametrelili modelin function calling desteği yetersizdir; regex hibrit yönlendirme bu eksikliğini başarıyla kapatmaktadır.
- SSE akıllı yanıtlar, tek seferlik HTTP isteklerine kıyasla kullanıcı deneyimini (algılanan gecikme) belirgin şekilde iyileştirmektedir.
- Bağlam penceresi taşmalar (context overflow) çok-dönümlü sohbette kritik bir tasarım kısıtı oluşturmaktadır; MAX_HISTORY_MSGS sınırı zorunludur.
- sim/real ikili modeli, donanım erişimi olmadan hızlı iterasyon döngüsü sağlaması ve geliştirme süresini önemli ölçüde kısaltmıştır.

11.3 Gelecek Çalışmalar

- Qwen2.5-3B veya Phi-3-mini ile kalite/hız karşılaştırması
- MQTT ile uzak sensör desteği ve çoklu cihaz yönetimi
- İnce ayar (fine-tuning) veya LoRA adaptasyonu ile domain-spesifik iyileştirme
- Kalibrasyon veritabanı ve sensör kayması (drift) otomatik düzeltme
- Mobil uyumlu PWA arayüzü ve anlık bildirim (push notification) desteği
- REST API dokümantasyonu ve OpenAPI şeması ile harici entegrasyon kolaylığı

11.4 Sonuç

Sensor, kısıtlı kaynaklara sahip gömülü sistemlerde yerel LLM entegrasyonunun pratik ve ölçülebilir biçimde hayata geçirilebileceğini kanıtlamıştır. C++ ile yazılmış verimli backend, sezgisel web arayüzü ve hibrit NLP katmanlı bir araya geldiğinde; internet bağlantısı ve bulut servisi gerektirmeyen, gizlilik odaklı bir IoT asistan ortaya çıkmaktadır.